Unbundled Transactions: A Study of a Layered Key-Value Store

BoardAI fixture paper, written for testing; public domain

Abstract

We describe a transactional key-value store that unbundles its parts: a transaction system that orders commits, a log system that makes them durable, and a storage system that serves reads. Each role runs as its own process and scales on its own. Transactions are serializable and use multi-version concurrency control with optimistic conflict checks. The store recovers from a failure by replacing the whole transaction system in a few seconds instead of repairing it. Every process is also tested inside a deterministic simulator that injects faults and replays any failure from a seed.

1 Introduction

Distributed databases usually bundle ordering, durability and storage into one kind of server. When one concern needs more machines, all of them get more machines. We take the opposite approach: each concern is a separate role, and the system is a pipeline between roles.

The design has three properties that matter for what follows. First, a single sequencer hands out versions, so every committed transaction has a place in one total order. Second, the log is the source of truth: a transaction is committed once its writes are durable on the log servers, before any storage server has applied them. Third, failures are handled by recovery as a whole rather than by repairing a single role in place.

Section 2 describes the roles. Section 3 follows one transaction through the commit path, and Section 4 describes recovery. Section 5 explains how the system is tested, and Section 6 reports measurements and lessons.

Unbundling is not free. A commit now crosses several machines, and each hop adds latency. The rest of this paper argues that the latency is small, predictable and worth paying, because each role becomes simple enough to reason about on its own, to scale on its own, and above all to test on its own.

We write for engineers who operate or build distributed databases. We assume familiarity with transactions, replication and consensus, but not with any particular system. Where a design choice has a well-known alternative, we name the alternative and say why we did not take it.

2 Architecture

Figure 1 (page 2) shows the roles. Clients link a library that talks to the cluster. The transaction system has three roles: the sequencer, which hands out read and commit versions; the proxies, which accept commits and batch them; and the resolvers, which check each transaction for conflicts. The log servers keep the durable log, and the storage servers keep the data, each

responsible for a range of keys. A small set of coordinators stores the cluster's configuration so that recovery can find it.

The roles never share memory or disks. A role finds the others through the configuration that the coordinators hold, and every message between roles carries the version it concerns, so a message from an older configuration can always be recognised and ignored.

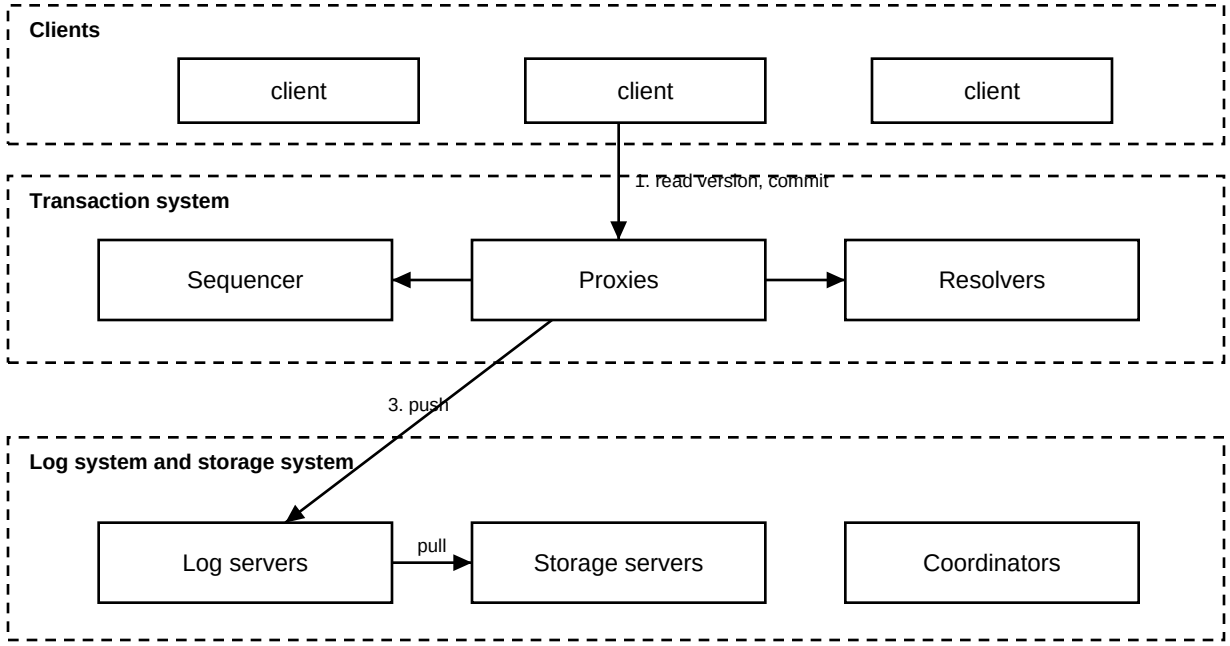

Figure 1: The architecture: clients, the transaction system (sequencer, proxies and resolvers), the log system and the storage system.

Table 1 lists each role, how many instances a typical cluster runs, and whether the role keeps state on disk. Only the log servers and the storage servers are stateful; every role of the transaction system can be replaced without losing data.

Storage servers are assigned key ranges by a data distribution process that watches their load and their free space. When a range becomes too hot or too large, it is split and one half is moved to another storage server. A move copies the data first and switches the assignment last, so readers never see a range without an owner.

Each key range is kept by three storage servers in a typical cluster, each in a different fault domain. A read goes to any one of them; a client library that sees a slow replica simply tries another one.

Clients cache the map from key ranges to storage servers. When the map changes, a storage server that no longer owns a range answers a read with a redirect, and the client refreshes its cache. The cache is therefore only a hint: correctness never depends on it, and a stale entry costs one extra round trip.

Versions are the glue between roles. The sequencer advances the version about a million times

per second of wall-clock time, so a version also tells roughly when a transaction committed. Storage servers keep a few seconds of old versions in memory, which is what lets a read at an older read version still find the values it should see.

Table 1: Roles in a typical cluster.

Role	Instances	Keeps state
Sequencer	1	no
Proxies	3 to 8	no
Resolvers	2 to 4	no
Log servers	3 to 12	yes
Storage servers	10 to 500	yes

3 Committing a Transaction

Figure 2 (page 3) shows the six steps of a commit. A client first asks a proxy for a read version; the proxy asks the sequencer for the latest committed version and returns it. The client then reads at that version directly from the storage servers and buffers its writes locally. Nothing is sent to the cluster until the client commits.

To commit, the client sends its read set, its write set and its read version to a proxy. The proxy obtains a commit version from the sequencer and sends the transaction to the resolvers. A resolver rejects the transaction when any key it read was written by a transaction that committed after its read version. Accepted transactions are pushed to the log servers, and the proxy replies to the client once every log server has made the batch durable.

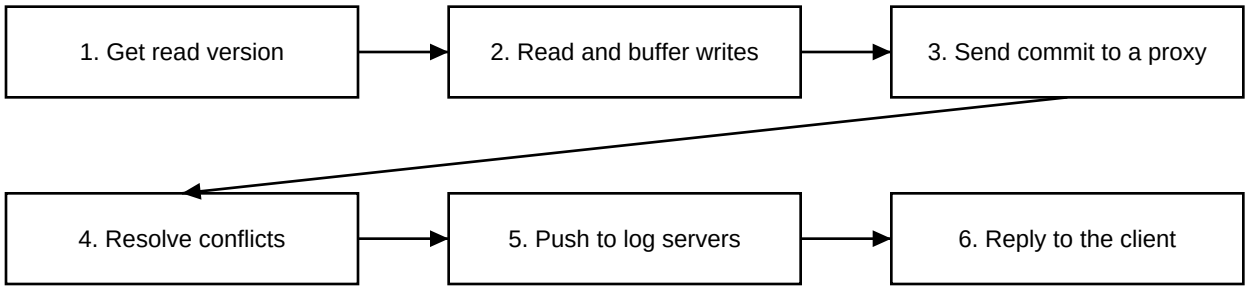

Figure 2: The commit pipeline, from the read version to the reply.

Because conflicts are only checked at commit time, read-only transactions never talk to the resolvers or the logs. They cost one round trip for the read version and then one round trip per read, which is why reads scale with the number of storage servers.

Proxies batch commits that arrive close together. A batch shares one commit version request, one resolver round and one push to the logs, so the cost of each hop is spread over many

transactions when the system is busy. When it is idle, a batch holds a single transaction and is sent at once.

Resolvers keep only a short history: the keys written by transactions committed within the last five seconds. A transaction whose read version is older than that window is rejected with a retryable error, which bounds the resolvers' memory and also bounds how long a transaction may run.

Read versions are served in batches too. A proxy that receives many requests for a read version in the same millisecond asks the sequencer once and answers them all, which keeps the sequencer, a single process, far from being a bottleneck. In our measurements the sequencer handled the load of a 24-machine cluster while using less than a tenth of one core.

Clients retry. A transaction that is rejected for a conflict, or that loses its proxy in the middle of a commit, is retried by the client library with a new read version. Applications therefore write transactions as functions that may run more than once, and must keep side effects outside them.

4 Recovery

When any role of the transaction system fails, the system does not try to repair it. Instead it runs recovery, the state machine in Figure 3 (page 4). A new sequencer reads the configuration from the coordinators, locks the old log servers so they accept no more pushes, recruits a new set of proxies, resolvers and log servers, and copies the tail of the old log that might hold committed but unapplied transactions. Then it starts accepting commits again.

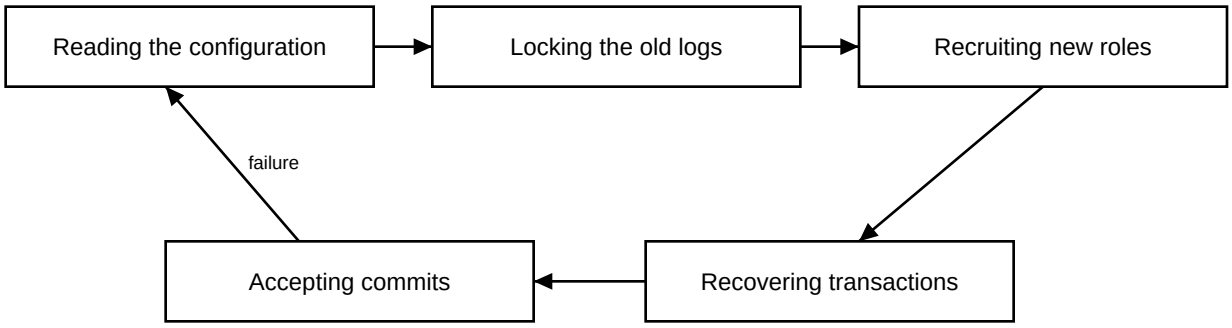

Figure 3: Recovery as a state machine; a failure in any state starts it again.

Recovery usually takes a few seconds. During that time no transaction commits, but reads at versions that were already committed continue to be served by the storage servers.

Locking the old logs is the step that makes recovery safe. Once a quorum of the old log servers has been locked, no proxy of the old generation can make another transaction durable, so the new generation can decide exactly which transactions were committed. Everything after that point is either copied forward or discarded.

A failure during recovery is not special. If a new role fails while recovery runs, recovery simply starts again from the first state with a newer generation number, as the failure arrow in Figure 3 shows.

5 Simulation Testing

Every role is written against an interface for the network, the disk and the clock. In tests these interfaces are replaced by a simulator that runs the whole cluster in one thread. The simulator chooses the order of every message and every timer from a random seed, injects failures such as dropped connections, slow disks and machine reboots, and checks the database's invariants after every step.

Because the simulation is deterministic, any failure it finds can be replayed exactly from its seed. Engineers run millions of simulated hours each night; most bugs are found there, long before they could reach a real cluster.

The simulator also speeds time up. A timer that would fire after ten seconds of real time fires as soon as nothing else is ready to run, so a simulated hour of a busy cluster takes a few seconds of real time. Tests can therefore explore rare interleavings, such as a machine rebooting in the middle of recovery, many thousands of times.

The price is discipline. Code must never read the real clock, spawn a thread or touch the network except through the simulated interfaces, and a lint enforces this. Third-party libraries are wrapped, or avoided, for the same reason.

Simulation does not replace testing on real machines; it changes what those tests are for. Real clusters still run nightly with real disks and networks, but their job is to catch performance regressions and problems in the operating system, not logic errors. Logic errors are found, reproduced and fixed in simulation, where a failing seed can be shared and replayed by anyone.

We also run the simulator with buggy versions of the code on purpose. A planted bug that the simulator fails to find within its nightly budget is a sign that the workload or the fault injection has a blind spot, and that blind spot is fixed before the planted bug is removed.

6 Evaluation and Lessons

Table 2 (page 5) reports throughput and latency for three workloads on a cluster of 24 machines. Commit latency stays below 20 milliseconds at the 99th percentile for small transactions. Read throughput grows almost linearly as storage servers are added, while write throughput is bounded by the log servers' disks.

Table 2: Throughput and latency by workload on 24 machines.

Workload	Reads/s	Writes/s	p99 commit (ms)
Read-heavy (90/10)	2,400,000	260,000	11
Balanced (50/50)	1,100,000	1,050,000	16
Write-heavy (10/90)	240,000	1,900,000	19

The main lesson is that unbundling makes each role simple enough to reason about and to test. The cost is latency: a commit crosses four roles. For the workloads we studied, that cost was small compared with what the simple recovery model and the simulator saved us.

A second lesson concerns operations. Because roles are stateless or nearly so, operators can replace machines freely: a misbehaving proxy is killed, and recovery recruits another one. The few stateful roles, the log servers and the storage servers, are where all the care goes.

A third lesson concerns limits. Every queue and every buffer between roles has a fixed size, and every role says what it does when one fills: it slows its clients down instead of growing without bound. Early versions without such limits behaved well in tests and then ran out of memory under a sudden burst of writes in production.

Finally, we would make the same choice again. Splitting the database into roles made each part small, made recovery a single well-tested path, and made the simulator possible. The latency we paid for it, a few milliseconds per commit, has never been the limit for the applications we built on it.